

第2回 Q&A まとめ

受講生から寄せられた24のテーマ。
すべてに共通するメッセージは——



この図解は**要点をまとめたもの**です。具体例やニュアンスなど、より詳しい内容は**動画本編**でご覧ください。

ツールは変わる。本質で判断する力を磨け。

どのツールが最強かではなく、なぜそうなのかを自分の頭で考える。



本質を理解する
なぜそうなのかを考える



仕組みで制御する

AIを思い通りに動かす

スキル＋ハーネス＋
レビュー



自分の頭で判断
鵜呑みにしない

24問を貫くキーワード

ここから先で、24の質問と回答をひとつずつ図解していきます。



全24問インデックス

01

初心者Cursor推奨の理由

本質で判断する力を育てる

02

Git・GitHubの始め方

リポジトリを作ってクローン

03

1人でもGitは有用か

「最終版_v2」から卒業する

04

GitをCursorからどう動かすか

AIと対話しながらコミット

05

スキルの置き場

プロジェクトスキルが基本

06

複数スキルの管理

責任の所在を考える

07

スキルのリファレンスの書き方

サブエージェントレビューの活用

08

ハーネスとは何か

AIを制御する仕組みの全体像

09

Cursor内のファイル保存場所

Git管理で変更を把握する

10

スキルを磨くと迷走の違い

チャットを区切る

11

「理解できないものを作るな」の…

大枠と処理の流れを理解する

12

スピードと品質のトレードオフ

突っ走ると結局遅い

13

AI-Driven Rules #5 vs #9

ピントの大きさが違う

14

コードレビューのやめどき

クリティカルな指摘だけ修正

15

レビュー結果のゴー・ストップ判断

わかりやすい言葉で返してもらう

16

自作ツールの修正沼

「ないとブチギレる人がいるか」

17 いい図解・悪い図解の基準
模範解答を作り込む

18 AIの出力するデザイン
Claude Designの可能性

19 AIフレンドリーな環境の全体像
GitHubリポジトリに情報を集約

20 ナレッジ管理ツールの選定
Notion・Obsidian・Git

21 APIキーの判断基準
家の鍵と同じように扱う

22 CopilotとPowerShellの業務自動化
AIの本質はツールに依存しない

23 AIの使い分け判断軸
常にフラットに見る

24 自律型AIエージェント（OpenCla…
批判的な目線で理解する

01 初心者Cursor推奨の理由

AI初心者にClaude CodeよりCursorがおすすめなのはなぜ？

Cursorを推す3つの理由



GUIの直感性

ファイル一覧・Git差分・ドラッグ&ドロップ。視覚的に理解しやすい



マルチモデル対応

Claude・Gemini・ChatGPT。その時々で最適なモデルを選べる



文章作成にも強い

マークダウンの編集・差分確認。開発だけでなくコンテンツ制作にも



ただし——

マコ自身は現在7〜8割**Claude Code**を使っている。Claude Codeが劣っているわけではない。CursorもClaude Codeも**同じモデル・同じ頭脳**で動いている。「**これが一番**」と安易に決めないことが大事。

📖 マコのひとこと

「本質的な能力を磨いていけばツールは関係ありません。どのツールを使うかで劇的な差がつくことはありません。誰かがいいとっていたから使うではなく、自分の頭と自分の経験で判断できるようになってほしいです」

02 Git・GitHubの始め方

重要性は理解できた。まず何から手を付ければいい？

最初の4ステップ

1

GitHubでリポジトリを作る（非公開でOK）

2

git cloneで手元に持ってくる

3

コミットしてみる

4

プッシュしてGitHubで確認

📖 マコのひとこと

「わからないことが出るたびに1つずつAIに聞いてください。私も毎日そうしています。重要性さえ理解すればあとは理解するだけです」

03 1人の開発でもGitは有用か

「最終版_v2」で事足りるのでは？

✗ ファイル名管理

企画書_最終版.docx

企画書_最終版_v2.docx

企画書_最終版_改良版.docx

企画書_本当の最終版.docx

どれが本物かわからなくなる

✓ Git管理

企画書.md (常に最新)

commit: 予算セクション追加

commit: 初稿作成

最新は1つ。過去にいつでも戻れる

📖 マコのひとこと

「削除してコミットすればいいんです。削除したコミットさえ残っていればまた戻すことができます。プロになればなるほど1人でもブランチを切って開発しています」

04 GitをCursorからどう動かすか

実際にCursorでどうGit操作しているか見たい

慣れてくるとAIとの対話でコミットするようになる

Cursorは変更ファイルの一覧やGit履歴をGUIで確認できる



99 マコのひとこと

「結局自分がどうやるんだろうと思ったときにAIにわかるまで聞いてみるのが最も効率よく学ぶ方法です。試行錯誤していくうちにAIで開発をするということ自体に慣れていきます」

[↑ インデックスに戻る](#)

05 スキルの置き場

ユーザースキルとプロジェクトスキル、どちらがいい？

基本はプロジェクトスキル

ユーザースキルは極力少なめに。迷ったら置かない。

スキルの置き場の構造イメージ

🌐 ユーザースキル

~/.claude/skills/

どこでも使える。ただし予期せぬ場所で動くリスクあり

📁 プロジェクトスキル

.claude/skills/

そのリポジトリ専用。必要な場所だけで動く

📦 モノレポ内スキル

tools/app-a/.claude/skills/

モノレポ内の特定アプリ専用。下の階層で開いて使う

📖 マコのひとこと

「使わないスキルがたくさんあるとこのプロジェクトではそのスキル使ってほしくなかったということがたくさん起きます。ホームディレクトリ直下のCLAUDE.mdも極力簡潔に書くのがおすすめです」

06 複数スキルの保存・管理

保存場所・更新先がわからなくなってきた



核心

スキルをどこにどう整理するか = **どこに責任を持たせるか**を考えること。これはエンジニアリングそのもの。スキルの読み込み方がCursorとClaude Codeで異なる点も理解するチャンス。

📖 マコのひとこと

「どこにスキルを置くのかどんな名前にするのかCursorでしか使わないのかClaude Codeだけでいいのか。それそのものが高度なエンジニアリングです」

07 スキルのリファレンスの書き方

リファレンスのまとめ方やAIレビューのプロンプトは？

おすすめプロンプト



最新のスキルのベストプラクティスについて熟知したAnthropicエンジニアならどうするか、サブエージェントレビューをしてください

スキルの教科書はAnthropicの公式ドキュメント。
「最新のベストプラクティスを熟知した」と入れることでウェブ検索を促す。
サブエージェントにすることで既存のコンテキストに引っ張られない。

📖 マコのひとこと

「Anthropicのエンジニアならどうするかということを私はよくレビューで使っています。スキルを作るスキルcreating-skillsも作っています」

08 ハーネスとは何か

CLAUDE.md • agents.md • hooksの関係と全体像

ハーネス = AIを制御する仕組みの総称

じゃじゃ馬なAIを安全に、思った通りに働かせるための道具一式



ルール

CLAUDE.md
agents.md



スキル

再利用可能な
専門能力



サブエージェント

コンテキストを
分離した担当者



Hooks

処理の前後に
自動実行

コアの考え方: エージェントがミスをするたびに、もう2度とミスをしないように
に**永遠に失敗しない仕組み**を整えていく

📖 マコのひとこと

「最初からハーネスのすべてを使おうとするのはオーバースペックです。でもAIの出力をより高めようとするのであればハーネスの考え方は間違いなく必要です」

↑ インデックスに戻る

09 Cursor内のファイル保存場所

作ったファイルが一体どこにあるのか不安

Cursorは**1つのフォルダーを開いてそこで作業する**設計。Git管理していれば何を変更したかが差分として見える。

⚠ **注意:** プロジェクト外のファイル（例: ユーザースキル）を変更した場合、Git差分には出てこない。Git管理下の場所だけで作業するのが安心。

🔗 マコのひとこと

「CursorのGitの画面でここで何が生まれたかを見てコミットしていく。どこの何を変更したかを意識しながら更新できます」

10 スキルを磨くと迷走の違い

やり取りを続けるとハルシネーションが起きる？

答え

チャットを区切る

1つのチャットですっとやり取りし続けるとAIのコンテキストウィンドウ・アテンションの限界でAIの性能が落ちていく。最初にいったことを守らなくなる。

× 迷走パターン

1つのチャットでスキルをずっと改善し続ける。指示がぶれ始め、前の修正を上書きし、方向性を見失う

✓ 磨くパターン

一旦ここまでと区切り、新しいチャットを開いてそちらで作業。これを繰り返してスキルを磨く

99 マコのひとこと

「ずっとやり取りし続けるとだんだん迷走していきます。最初にいったことを守らなくなっていくます。なので区切らなきゃいけないんです」

11

「理解できないものを作るな」の守り方

コードがほぼ理解できない。どの程度理解すべき？

今はこれで良い。もっと理解できるようになる。



Cook Doのたとえ

合わせ調味料の中身をすべて理解しなくても美味しい料理は作れる。理解できるに越したことはないが、すべてを理解しないと作れないわけではない



車の運転のたとえ

エンジンの仕組みをすべて理解していなくても運転はできる。大事なのは**処理の流れ**を理解すること。材料を切る→炒める→味付けのような手順



さらに一步進むなら

AIがやったことを**疑ってみる**。中学生でもわかるように説明してもらい、「もっとこうしたほうがいいんじゃないの？」とぶつけてみる。実はそれが正しく鋭い指摘であることはよくある。

🔗 マコのひとこと

「事細かには理解しなくてもバグが起きないような仕組みや構造を作る。それさえできればすべてを理解する必要はないんです」

12 スピードと品質のトレードオフ

最初はスピードを捨てて理解に徹するのが正解？

目的によって切り替える

🚀 仮説検証フェーズ

価値があるかわからない段階。多少コードの品質は妥協してもOK。まずフィードバックをもらう

💎 高品質プロダクト

使い続けるプロダクト。整理整頓しながら作る。パッチワークはすべて負債になる



教訓: 突っ走ると結局遅い

コードを書いたあとにエンジニアサブエージェントにレビューさせる。クリティカルな指摘を無視すると大きなやり直しやバグの温床になり、トータルでは高いコストを払う。

🔗 マコのひとこと

「AIに運転を全任せしたら見かけ上のスピードは増えるけどたまに事故ったときにどうしようもできなくなる。ちゃんと自分でハンドルを握って理解しながら運転を続けられれば事故らずハイスピードで運転できるようになっていきます」

13 AI-Driven Rules #5 vs #9

「プロトタイプを作れ」と「スピードを出すな対話しろ」は矛盾？

ピントの大きさが違う

🔍 #5 大きなピント

「議論するよりプロトタイプを作れ」

企画書を延々と作るよりも、動くものを見せたほうが物事が前に進む。AI時代は開発のハードルが下がった。

🔍 #9 小さなピント

「スピードを出すな。対話しろ」

プロトタイプを作る中でも、何が起きているかわからないまま突き進むのはNG。AIと対話しながら理解して進める。

📖 マコのひとこと

「スピードを出さないで対話をしたことによって自分の理解が深まって、次はもっとスピードを出せるようになっていく。もしそれを省略してしまったらずっとわからないままです」

14 コードレビューのやめどき

AIのレビューは無限に改善案を出してくる。どこで線を引く？

2つのタイミングでレビューする

1

開発計画のレビュー

コードを書く前に計画段階でシニアエンジニアとUI/UXデザイナーのサブエージェントにレビューさせる。コードを書く前のほうが効率がいい

2

フェーズごとのコードレビュー

フェーズ単位で書いたコードをレビュー。**クリティカルな指摘だけ修正**して終わり。それ以上何回もやらない

99 マコのひとこと

「かなり細かいそこまでやるかという指摘もしてきます。噛み砕いて説明してもらってこれ本当にやるべきかを自分の頭で理解して、本当に今やるべきだと思ったものはやります」

15

レビュー結果のゴー・ストップ判断

初心者はゴー・ストップの判断がつかず、そのまま実行してしまう

3つのおすすめ

- 1 **わかりやすい言葉で返してもらう** —— プロンプトに「技術がわからない人でも直感的にわかるように簡単な言葉で説明して」と入れておく
- 2 **わからないときは焦らず質問する** —— 全くわからないまま進めると意図しない方向に進むことがある
- 3 **繰り返しの指示はスキルやCLAUDE.mdに入れる** —— 毎回聞き返さなくて済むようにする

📖 マコのひとこと

「正直私もなんか問題ありそうだから修正しとこうって進めていることぶっちゃけあります。ただすごく罪悪感を抱えながらやっています」

16 自作ツールの修正沼

細部の修正に沼ってしまう。どこまでやるかの線引きは？

判断基準

「ないとブチギレる人がいるか？」

必要か必要じゃないかと考えると、作り手は必要だと考えてしまう。
「これがないと使いものにならなくなるのか」という基準で判断する。

📖 マコのひとこと

「ちょっと機能を付け足したときに思っている5倍10倍時間がかかって沼にはまるってことは本当によく起きます。完璧を目指さず手動対応を残すほうが現実的な場面もあります。というかそのほうが多いです」

↑ インデックスに戻る

17 いい図解・悪い図解の基準

良し悪しの判断に迷う。再現性を持たせたい

答え

模範解答を作り込む

模範解答を作る以上にAIの性能を上げる方法はない



いい図解の条件

丁寧すぎるぐらいがちょうどいい。知識がある人がちょっと冗長だなと感じても、そのコストは小さい。知識のない人が「これってどういうこと？」と質問して回答する工数のほうが圧倒的に生産性を下げる。

📖 マコのひとこと

「模範解答を作り込むことです。模範解答を作る以上にAIの性能を上げる方法はないといってもいいぐらい模範解答命です」

AIに平均点ではないオリジナリティのあるデザインを出してもらうには？

答え

Claude Designを試してみしてほしい

デザインシステムの土台を作れる。デザインの微調整インターフェースも用意されている。デザインの壁を破ってきた感覚がある。

99 マコのひとこと

「本当にデザイナーがディテールまで凝ったレベルのものを作るのは非常に難しかった。でもClaude Designはそのデザインの壁を破ってきた感覚があります」

ストック情報の置き場やツールの使い分けは？

GitHubリポジトリに情報を集約



GitHubリポジトリ

SSoT。整理整頓されたコード・ドキュメント・コンテキスト



Linear

タスク管理。AI経由で操作しやすいから採用



Supabase / クラウドストレージ

大量データ・画像・動画。リポジトリに向かない大容量データ用

Notionは以前使い倒していたが今はほぼ使っていない。ただし使ってはいけないツールではない

🔗 マコのひとこと

「Git上にコンテキストを整理しておくとそのままスキルを呼び出して最新のコンテキストを基にAIに仕事させることができます」

20 ナレッジ管理ツールの選定

Notion・Obsidian・Git、どれがいい？

圧倒的にこれがいいというツールは現時点で存在しない

個人の好みと使い方で選ぶ

Notion

高機能で使いやすいが、プレーンなデータではなくなる。AIにとって使いにくい面もある

Obsidian

シンプルなドキュメント管理。マークダウンベース。ただし整理整頓を続けられるかが鍵

Git (マコの選択)

整理整頓された情報をストック。差分管理ができる。ただしメモ帳としては不向き

📖 マコのひとこと

「プレーンなデータがほしいんです。純粋なマークダウンファイルとして保存したい。Notionは便利だけど、そのままではAIが読みやすいデータではないんです」

↑ [インデックスに戻る](#)

21 APIキーの判断基準

安全かどうかAIの回答が割れる。最終的にどう判断する？

APIキー = 家の鍵

粗末に扱ってはいけない

❌ 絶対NG

課金情報が紐づいたAPIキーをGitにコミットする。万が一コミットしたらすぐにAPIキーを無効化する

✅ マコの方法

すべて1Passwordで管理。開発サーバー起動時も1Passwordから都度読み込む仕組みにしている

📖 マコのひとこと

「おそらくNGといっているほうはかなり気にしすぎのレベルになっているんじゃないかなっていうのが私の経験則です。でもちゃんと精査してください」

22 CopilotとPowerShellの業務自動化

会社でCursorが使えない。マイクロソフト環境でAI活用は可能？

結論

AIの本質はツールに依存しない

どの言語でもプログラミングの原理原則は変わらない。どのツールでもAI活用の本質は変わらない。最初にどのツールを選ぶかですべてが決まるわけではない。

📖 マコのひとこと

「1つのツールがないと活躍できない。そんな人を育てたいのではありません。どんなツールに変わろうがテクノロジーの本質、AIの本質は変わりません」

23 AIの使い分け判断軸

真子さんの使用目的別の基準は？

マコの現在の使い分け（常に変わる前提）



ツール・アプリ開発

Claude Code



文章作成

Cursor上のClaude



調べもの

Claude



画像生成

Gemini



PC操作・定型作業

Claude Cowork

重要: これは収録時点の話。来週には変わっているかもしれない。常にフラットに見て、「これが1番」と決めつけない。

99 マコのひとこと

「もうこれはオワコンだみたいなことを考えないようにしています。常にフラットに見て、今のところはこうですということが私の伝えられる限界です」

24

自律型AIエージェント（OpenClaw）

最近話題の自律型AIエージェントについてどう思う？

やや懐疑的。バズが先行している印象。

自分専用の秘書が作れるような体験は面白い。ただし実用レベルで使うにはセキュリティ面も含めてまだ難易度が高い。



新しい技術との向き合い方

新しいツールが出たとき、**一定批判的な目線**を持って理解する。「結局何なのか」「何がすごいのか」「どういう仕組みなのか」「なぜ他ではできないのか」——安易に使える・使えないの判断を下さないことが重要。

99 マコのひとこと

「どんな仕組みなのかを理解して単純に使える使えないという判断を安易に下さないことが重要だと考えています」

[↑ インデックスに戻る](#)

